Studio Nx

Backend(FastAPI/Python) 45 — change_password()

2026-10-12

Backend(FastAPI/Python) 45 — `change_password()`

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-09 비밀번호 변경·탈퇴: 비밀번호 변경 (PRD 01 FR-107)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	44 `check_current_password()` · 46 `withdraw()`

1. 하는 일

지금 비밀번호를 확인한 다음, 새 비밀번호가 규칙에 맞으면 새 해시로 바꿔 저장합니다.

지금 비밀번호가 `Abcdef1!` 일 때 보낸 값 (지금 / 새)	결과
`Abcdef1!` / `Newpass9#`	저장합니다. 이후로는 `Newpass9#`로만 로그인할 수 있습니다
`Abcdef1?` / `Newpass9#`	400 `WRONG_PASSWORD` "Password is incorrect." (44)
`Abcdef1!` / `abcdefgh`	422 "Check the password requirements."와 `details: {"new_password": ["upper", "number", "special"]}`
`Abcdef1!` / `Ab1!`	422와 `["length"]`
`Abcdef1!` / `Abcdef1!` 한글	422와 `["allowed"]`
`Abcdef1!` / `Abcdef1!` (같음)	422 **"Use a different password."**와 `details: {"new_password": "same_as_current"}`
다른 회원이 `Newpass9#`를 쓰고 있음	상관없이 저장합니다

오류가 나면 DB의 해시는 그대로 남습니다.

2. 언제 쓰나

쓰는 곳	
`PUT /users/me/password` (47)	userInfo.html의 비밀번호 칸 3개(지금, 새, 확인)를 채우고 Change password를 누를 때 씁니다 (P1-10)

사용자 결정 (2026-10-07)

새 비밀번호가 지금 비밀번호와 같으면 막습니다. 템플릿에는 "(선택)"으로 적혀 있었습니다.

- 이것은 **같은 회원 안에서** 비교하는 것입니다. 다른 회원과 비밀번호가 겹치는지는 검사하지 않습니다. 겹쳐도 문제가 없고, 오히려 검사하면 "누군가 이 비밀번호를 쓰고 있다"는 단서가 되기 때문입니다. 또 해시마다 소금(salt)이 달라서 비교할 수도 없습니다.
- 예전에 쓰던 비밀번호들까지 막는 기능(비밀번호 이력)은 범위 밖입니다.

3. 코드

import (파일 맨 위)

```
from app.core.security import check_password_rules, hash_password, verify_password
```

`check_password_rules`와 `hash_password`를 이번에 추가했습니다 (44번에서 추가한 `verify_password` 줄에 이어서).

함수

```
def change_password(db: Session, user: User, current_password: str, new_password: str) -> None:
    check_current_password(user, current_password)
    failed = check_password_rules(new_password)
    if failed:
        raise AppError(
            ErrorCode.VALIDATION_ERROR,
            422,
            "Check the password requirements.",
            details={"new_password": failed},
        )
    if verify_password(new_password, user.password_hash):
        raise AppError(
            ErrorCode.VALIDATION_ERROR,
            422,
            "Use a different password.",
            details={"new_password": "same_as_current"},
        )
    user.password_hash = hash_password(new_password)
    db.commit()
```

문법 풀이

import 문법	뜻
`check_password_rules`, `hash_password`	P1-04(14, 15)에서 만든 함수입니다. 가입과 같은 규칙으로 검사하고 해시를 만듭니다

식	뜻
`if failed:`	빈 목록이면 거짓이고, 실패한 조건이 하나라도 있으면 참입니다
`verify_password(new_password, user.password_hash)`	새 비밀번호를 **지금 해시**에 대어 봅니다. 맞으면 지금 비밀번호와 같다는 뜻입니다
`-> None`	돌려줄 값이 없습니다. 라우터는 204(본문 없음)로 응답합니다

4. 흐름

1. 지금 비밀번호를 확인하고, 틀리면 400으로 응답합니다.
2. 새 비밀번호의 규칙을 검사하고, 틀리면 422와 실패한 조건을 돌려줍니다.
3. 지금 비밀번호와 같으면 422로 응답합니다.
4. 새 해시로 저장합니다.

5. 왜 이렇게

선택	이유
지금 비밀번호를 **먼저** 확인	다른 사람이 내 자리에서 새 비밀번호 규칙을 시험해 보는 것조차 못 하게, 본인 확인을 가장 먼저 합니다
규칙 검사를 "같음" 검사보다 먼저	지금 비밀번호는 이미 규칙을 통과했던 값이라서 순서가 결과를 바꾸지 않습니다. 그래서 DB를 보지 않는 가벼운 검사를 앞에 둡니다 (P1-04와 같은 원칙)
"같음" 비교에 `verify_password` 사용	서버에는 지금 비밀번호의 원문이 없으므로, 해시에 대어 봐야 알 수 있습니다. 지금 비밀번호 칸의 값과 글자로 비교(`current == new`)할 수도 있지만, 해시와 비교해야 "실제로 저장된 것과 같다"는 것이 확실해집니다
`details`의 칸 이름을 `new_password`로 함	화면(P1-10)이 그 칸 아래에 문구를 붙일 수 있게 했습니다. 요청 본문의 칸 이름과 같습니다
`refresh`를 하지 않음	돌려줄 정보가 없기 때문입니다 (응답 204)

알아 둘 점 (다른 기기): 비밀번호를 바꿔도 다른 브라우저에 이미 있는 리프레시 쿠키(최대 3일)는 그대로 동작합니다. 리프레시 토큰을 DB에 저장하지 않는 설계이기 때문입니다 (P1-05 34번 §5와 같은 한계). 필요해지면 "이 시각 이전에 만든 토큰은 무효"라는 값을 회원 행에 두는 방법이 있습니다 (토큰의 `iat`, P1-05 26).

6. 시험 (`tests/test_user_service.py`에 7개를 추가해서 모두 70개, 실제 MySQL로 실행)

시험	확인한 것
바꾸기	DB 해시가 바뀌고, 새 비밀번호는 맞고 옛 비밀번호는 틀립니다
지금 비밀번호가 틀림	400 `WRONG_PASSWORD`이고, 해시는 그대로입니다
규칙 3가지	약한 비밀번호, 짧은 비밀번호, 한글과 공백은 422이고, `details.new_password`에 실패 목록이 담깁니다
지금과 같음	422 "Use a different password."와 `same_as_current`이고, 해시는 그대로입니다
다른 회원과 같은 비밀번호	저장됩니다

전체 시험 328개가 통과했고, `ruff` 검사도 통과했습니다.